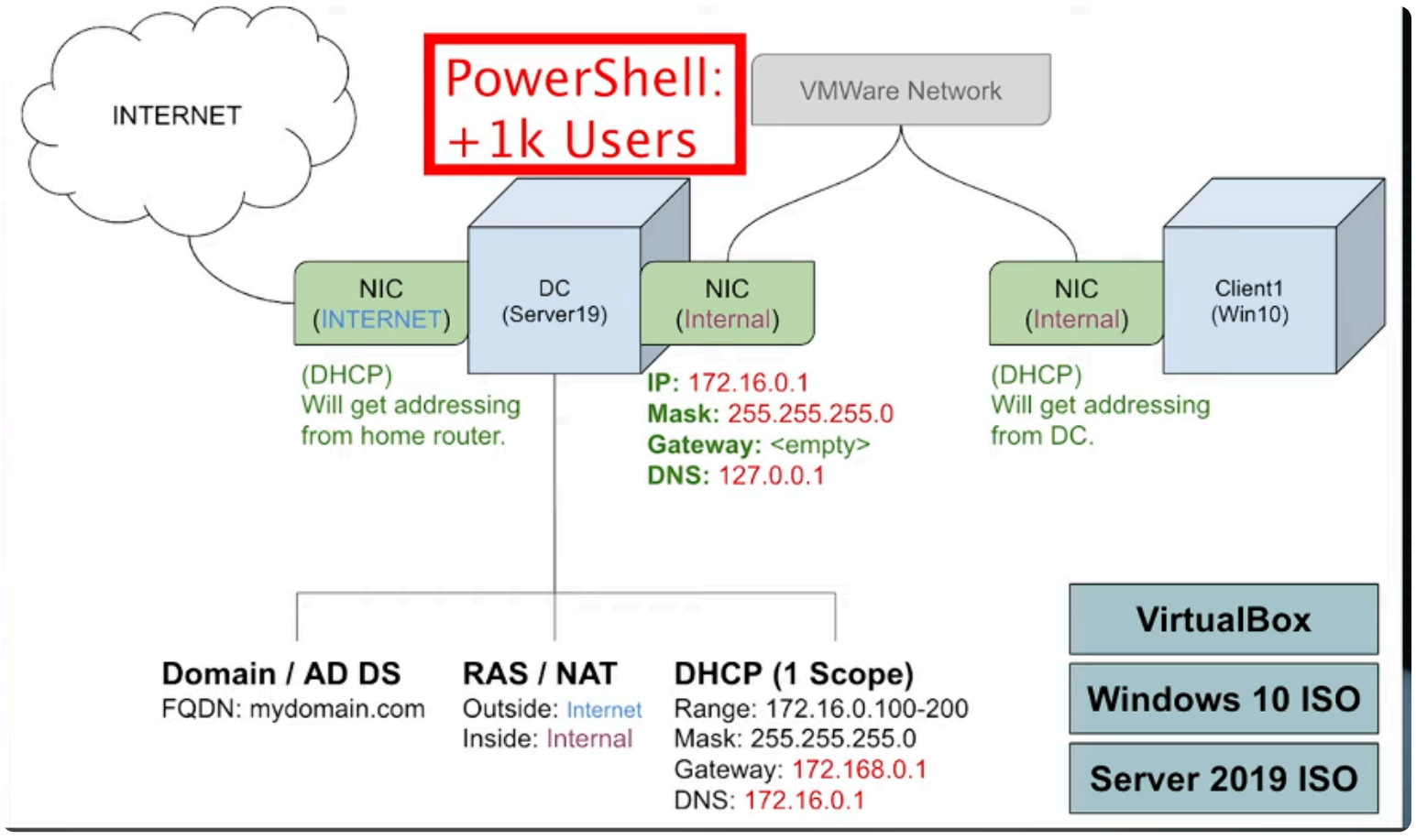
HOME LAB 1

Step 1: Active Directory Setup

Reference: How to Setup a Basic Home Lab Running Active Directory (Oracle VirtualBox) | Add Users w/PowerShell - YouTube



1. Windows Server Installation

Objective: Deploy a Windows Server 2019 instance to serve as the Domain Controller (DC) for the TradeGuard lab.

VM Configuration

Setting	Value
VM Name	`TradeGuard-DC`
Base ISO	Windows Server 2019 Evaluation
Virtual Disk	60 GB
Memory (RAM)	4 GB
CPU Cores	1
Password	`#Ckhandor123`
Network	Dual Adapters: NAT (Internet) + Host-Only (Internal Lab)

2. Network Configuration

We configure two network adapters to ensure:

- The server can access the internet (for updates and package installs)
- The internal lab VMs (Kali, Ubuntu SIEM, Windows clients) can communicate securely with the DC.

Adapter	Type	Purpose	Example IP	Notes
Adapter 1	NAT (`VMnet8`)	Provides internet access through host	Dynamic	Used for Windows updates and downloads
Adapter 2	Host-Only (`VMnet1`)	Internal lab network	`172.16.0.1`	Used for AD, DNS, and internal VM communication

Network Verification

After installation:

1. Open **Command Prompt** → Run:

   ```powershell
   ipconfig /all
   ```

   Ensure both adapters appear (NAT + Host-Only).
2. Set a **static IP** for the internal adapter (`Ethernet1`):
   - IP: `172.16.0.1`
   - Subnet mask: `255.255.255.0`
   - Default gateway: *(leave blank or use `127.0.0.1`)*
   - DNS server: `127.0.0.1` *(will later point to local DNS once AD DS is installed)*
3. Verify connectivity:

   ```powershell
   ping 8.8.8.8          # Internet connectivity via NAT
   ping 172.16.0.1   # Loopback / local connectivity
   ```

3. System Configuration Summary

Parameter	Value
Hostname	`TRADEGUARD-DC`
IP Address	172.16.0.1
Domain	`tradeguard.com`
Default Gateway	*(None – DC will serve as DNS)*
DNS Server	`127.0.0.1`
Admin Account	`Administrator`
Password	`#Ckhandor123`

4. (Optional) Disable Firewall Temporarily

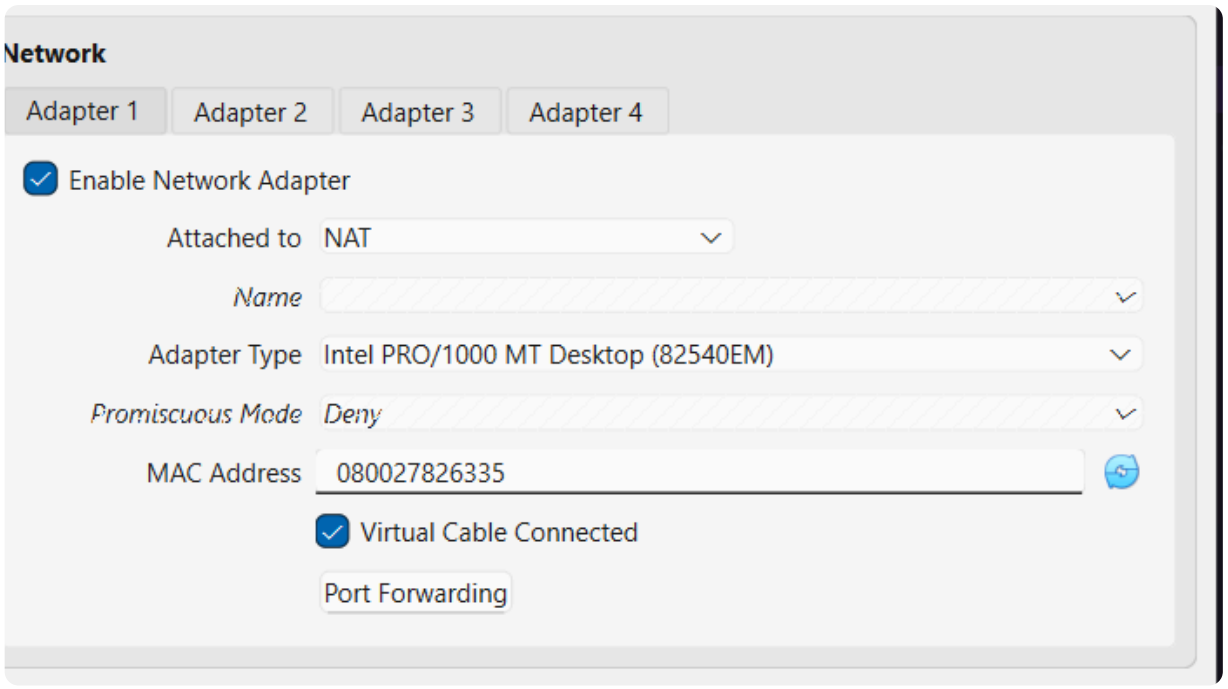
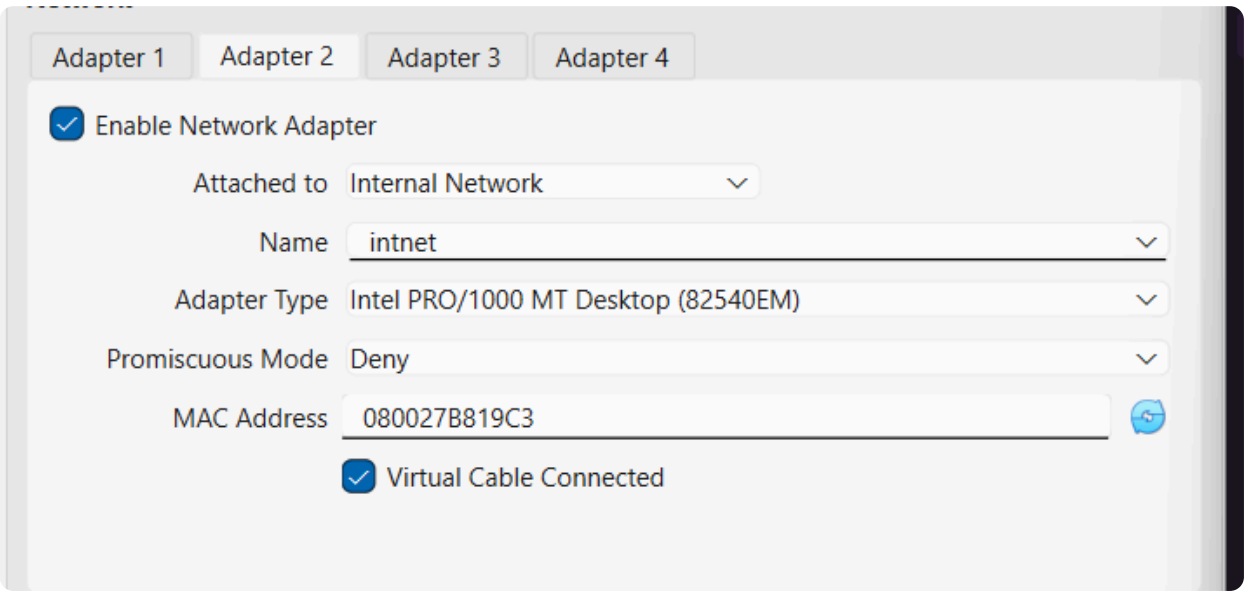
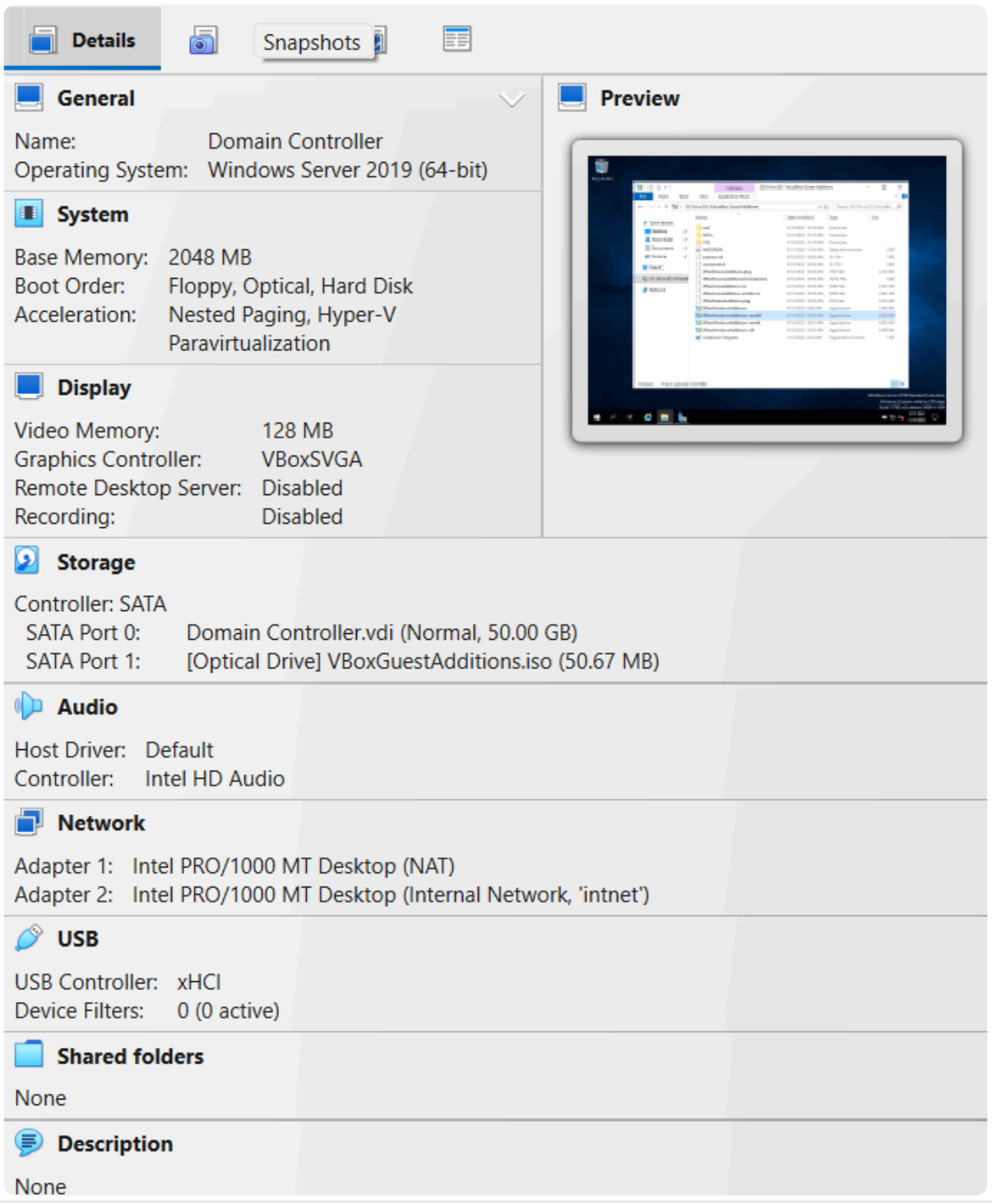
To simplify testing and avoid connectivity issues during setup:

1. Open **Windows Defender Firewall → Turn Windows Defender Firewall on or off**
2. Turn **off** for:
   - Domain Network
   - Private Network
   - Public Network

5. Pre-Setup Validation

Before installing **Active Directory Domain Services (AD DS)**:

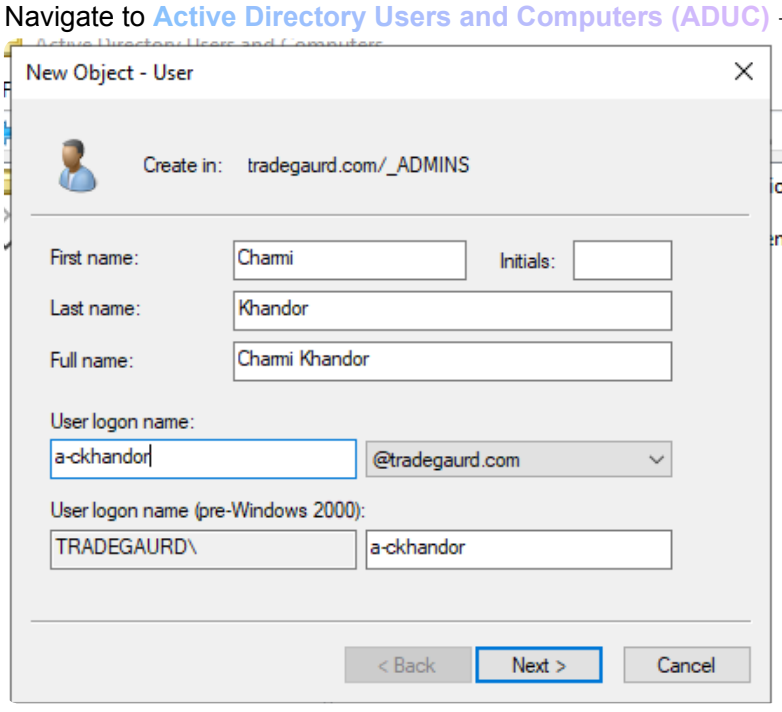
Renamed the server → `TRADEGUARD-DC`
Set static IP → `172.16.0.1`
Verified NAT and internal connectivity
Took VM snapshot → *"Pre-AD Setup"*

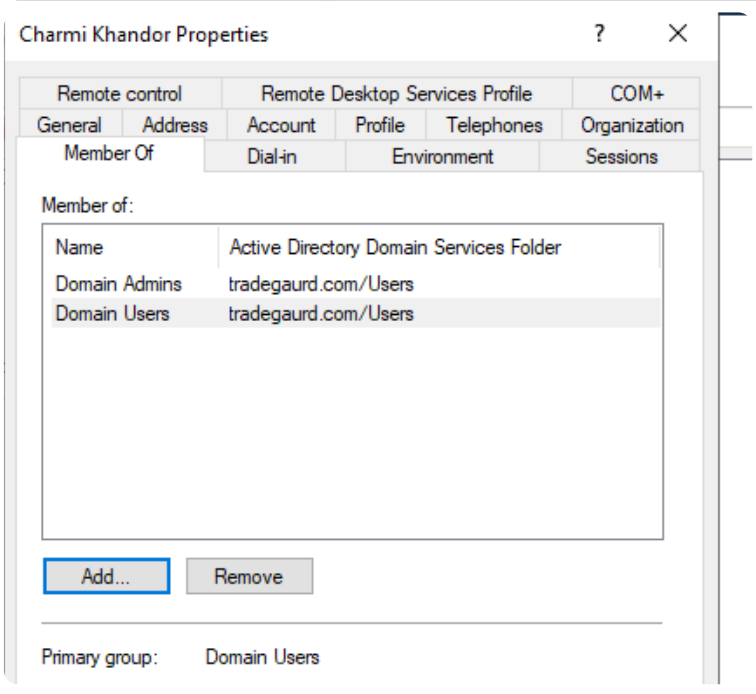
Description	Example
showing both adapters	
Network Adapter settings	
Server Manager overview	

1. Download and Configure Active Directory Domain Services (AD DS)

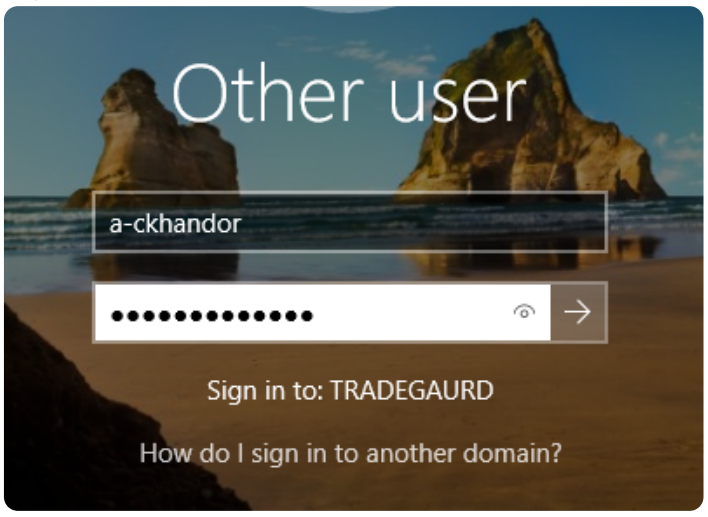
Steps:

1. **Set up the Domain Controller (DC)**
   - **Domain Name:** `tradegaurd.com`
   - **Password for the default admin:** `#TradeGaurd123`
   - During setup, make this server the **default domain controller** for the domain.
2. **Create a Custom Domain Admin Account**
   - Instead of using the default administrator account, create a custom domain admin.
   - Navigate to **Active Directory Users and Computers (ADUC)** → **Organizational Units (OUs)** → Create a new OU named `_ADMINS`.





   - Inside `_ADMINS`, create a new user.
     - **Password:** `Password1`
   - Make this user a **domain admin**.
   - Sign in with the new domain admin account.



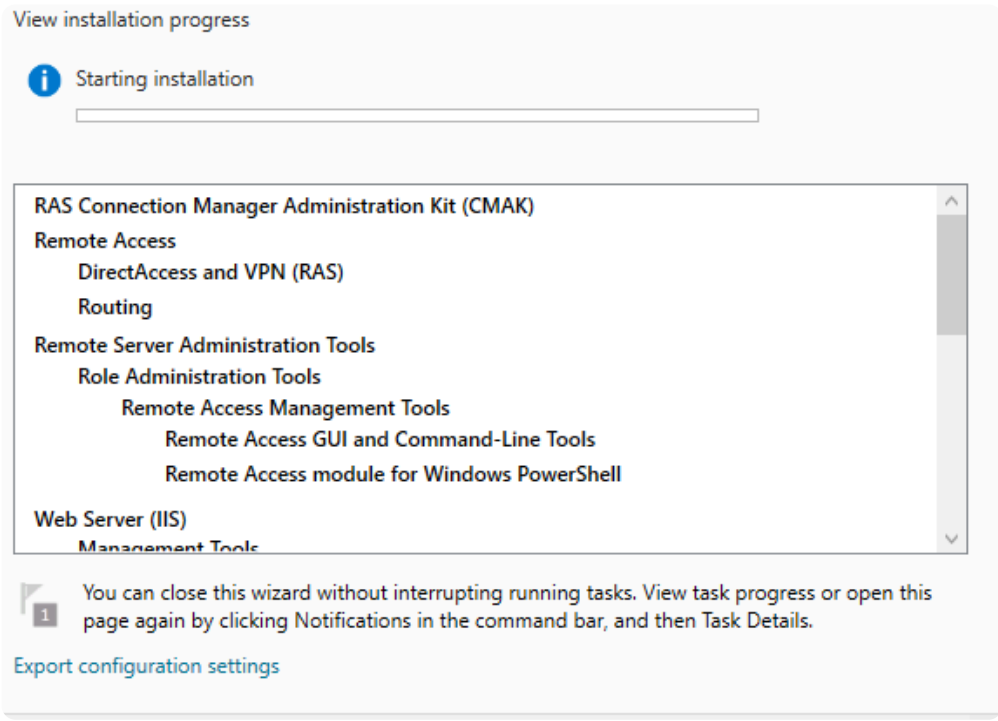
Explanation:
Using a separate domain admin account improves security and avoids using the default admin account, which is a common attack target.

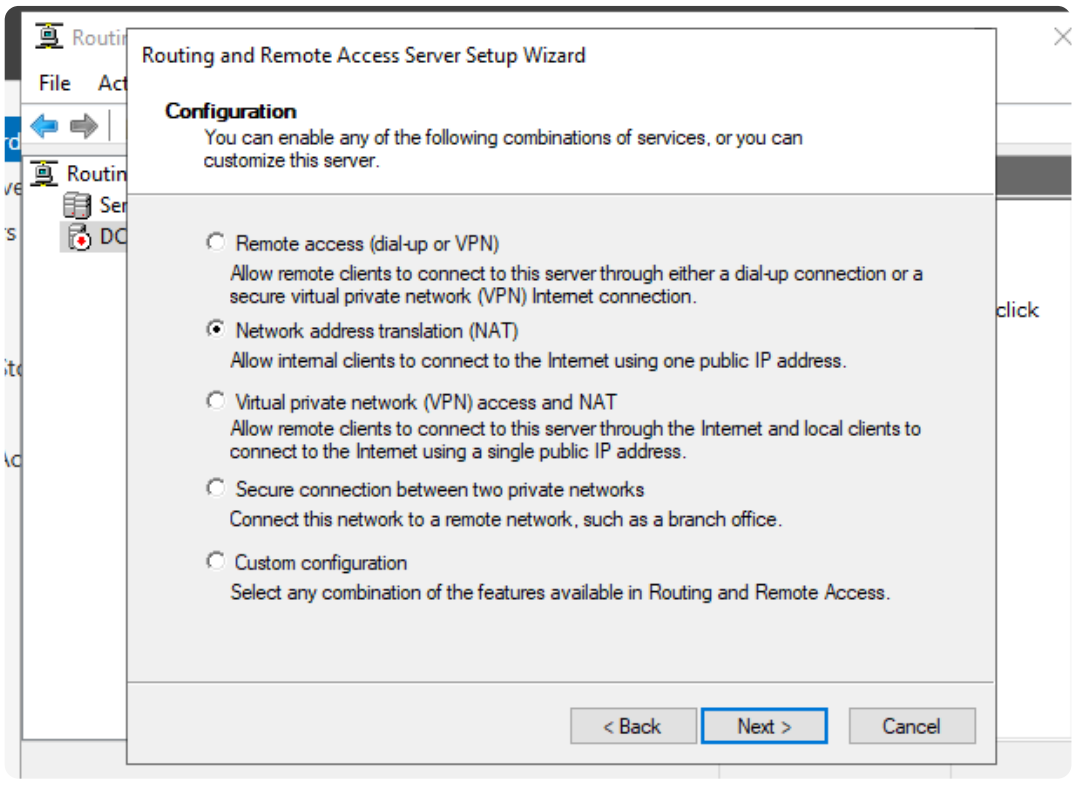
2. Install Routing and Remote Access Service (RAS/NAT)

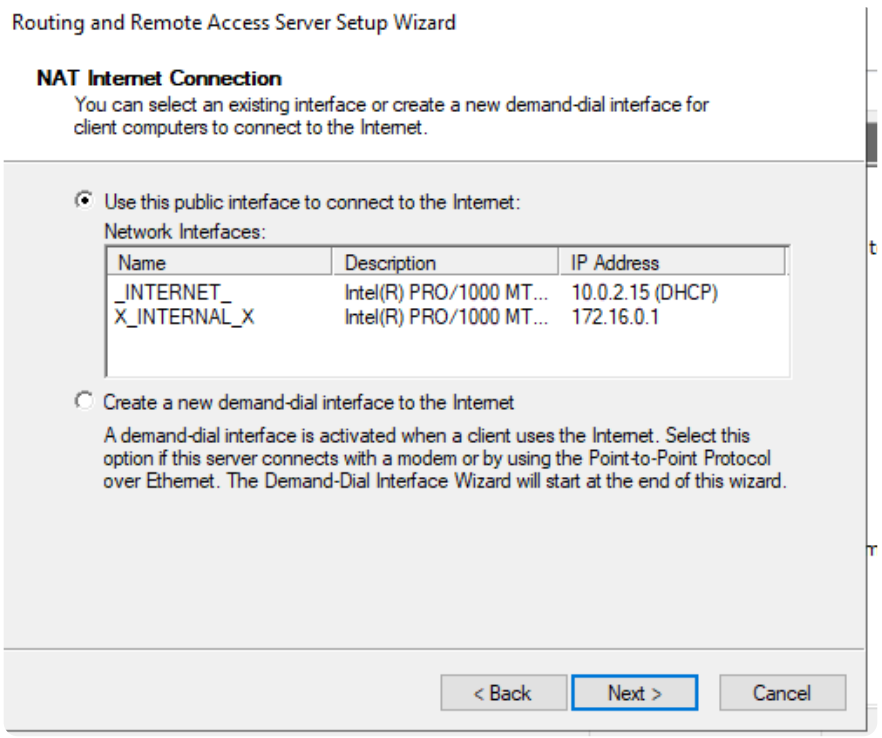
Purpose:
Allows Windows 10 clients on the internal network to access the internet through the virtual network.

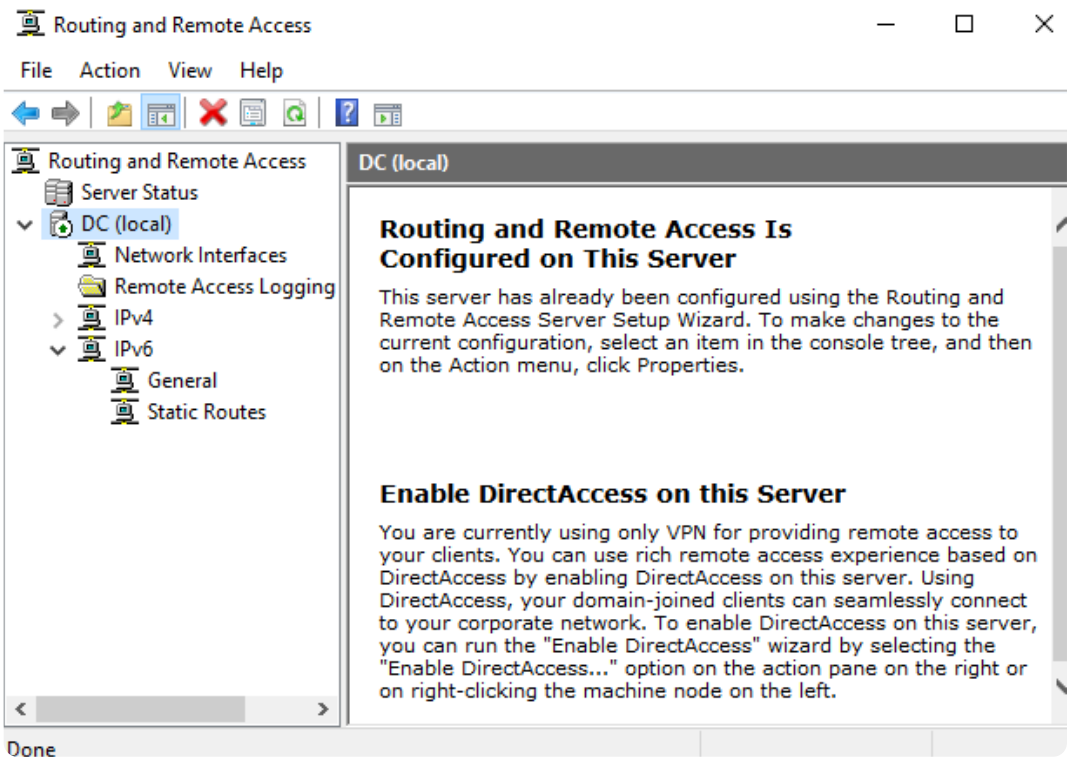
Steps:

1. Open Routing and Remote Access tool.
2. Configure NAT (Network Address Translation) for the internal network.









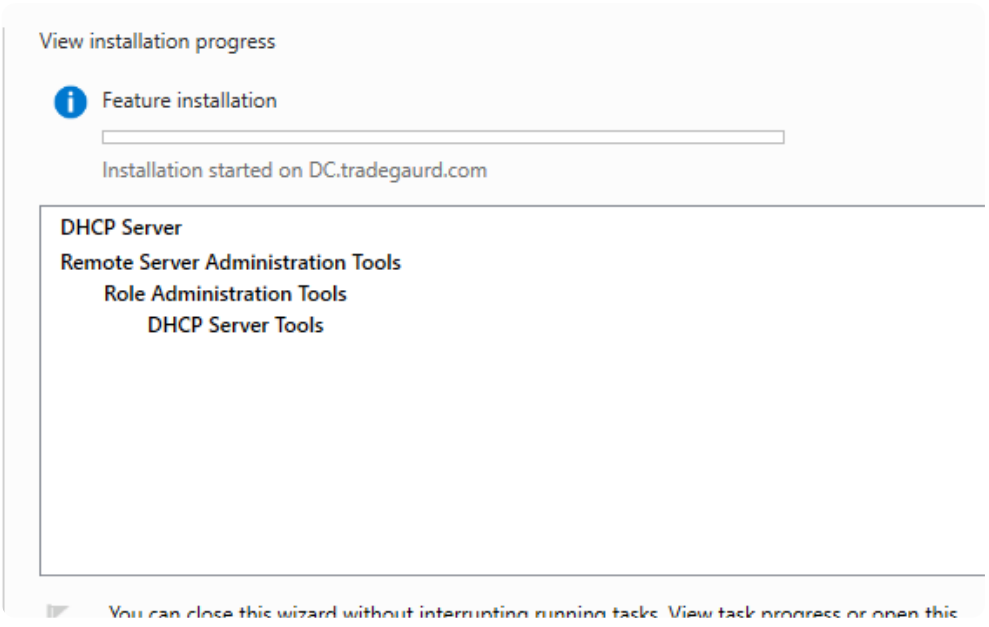
Explanation:
NAT allows multiple clients on a private network to share a single public IP address for internet access. This is essential for testing virtual clients while keeping them isolated from external networks.

3. Set Up DHCP Server on the Domain Controller

Purpose:
Automatically assign IP addresses to clients on the internal network so they can communicate internally and access the internet.

Steps:

1. Open **DHCP Server** in the server manager.



2. Create a new **DHCP scope**:
   - **Range:** `172.16.0.100 – 172.16.0.200`

- **Subnet Mask:** `255.255.255.0`
- **Router IP:** Use the domain controller's IP (since NAT is configured)
- **DNS Server:** Domain Controller
- No exclusions needed
- Lease duration can remain default for lab purposes

Explanation:
DHCP automates IP configuration, preventing conflicts and simplifying network management. Assigning the DC as the router and DNS ensures all internal clients route traffic properly and resolve domain names.

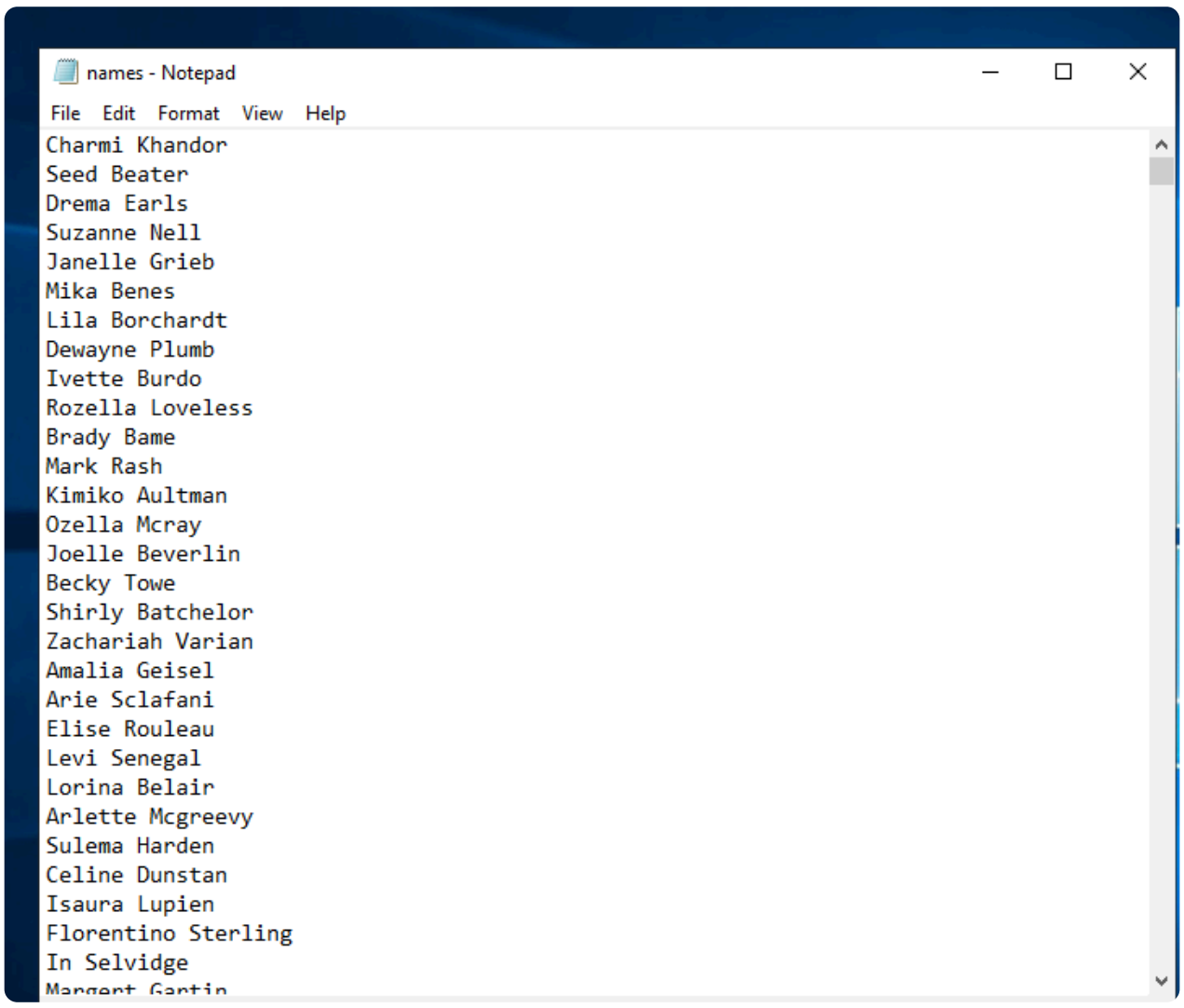
4. Create Users Using PowerShell

Purpose:
Automate bulk user creation in Active Directory with consistent credentials and organizational structure.

Steps:

1. **Prepare a text file** `names.txt` with user names, one per line:

```
John Doe
Jane Smith
```



2. **Set Execution Policy** to allow scripts:

```powershell
Set-ExecutionPolicy Unrestricted
```

3. **Run PowerShell Script**:

```powershell
# Password to assign to all users
$PASSWORD_FOR_USERS = "Password1"

# Get list of users from text file
$USER_FIRST_LAST_LIST = Get-Content .\names.txt

# Convert password to a secure object that AD can use
$password = ConvertTo-SecureString $PASSWORD_FOR_USERS -AsPlainText -Force

# Create a new organizational unit called _USERS
New-ADOrganizationalUnit -Name _USERS -ProtectedFromAccidentalDeletion $false

foreach ($n in $USER_FIRST_LAST_LIST) {
    $first = $n.Split(" ")[0].ToLower()
    $last = $n.Split(" ")[1].ToLower()
```

```
    # Username format: first letter of first name + last name
    $username = "$($first.Substring(0,1))$($last)".ToLower()
    Write-Host "Creating user: $($username)" -BackgroundColor Black -ForegroundColor Cyan

    # Create AD user
    New-AdUser -AccountPassword $password `
               -GivenName $first `
               -Surname $last `
               -DisplayName $username `
               -Name $username `
               -EmployeeID $username `
               -PasswordNeverExpires $true `
               -Path "ou=_USERS,$(([ADSI]`"").distinguishedName)" `
               -Enabled $true
}
```

Explanation:

- Automates creation of multiple users.
- Sets a secure, consistent password.
- Organizes users in `_USERS` OU for better management.
- Usernames follow a predictable format (`first initial + last name`), simplifying login conventions.

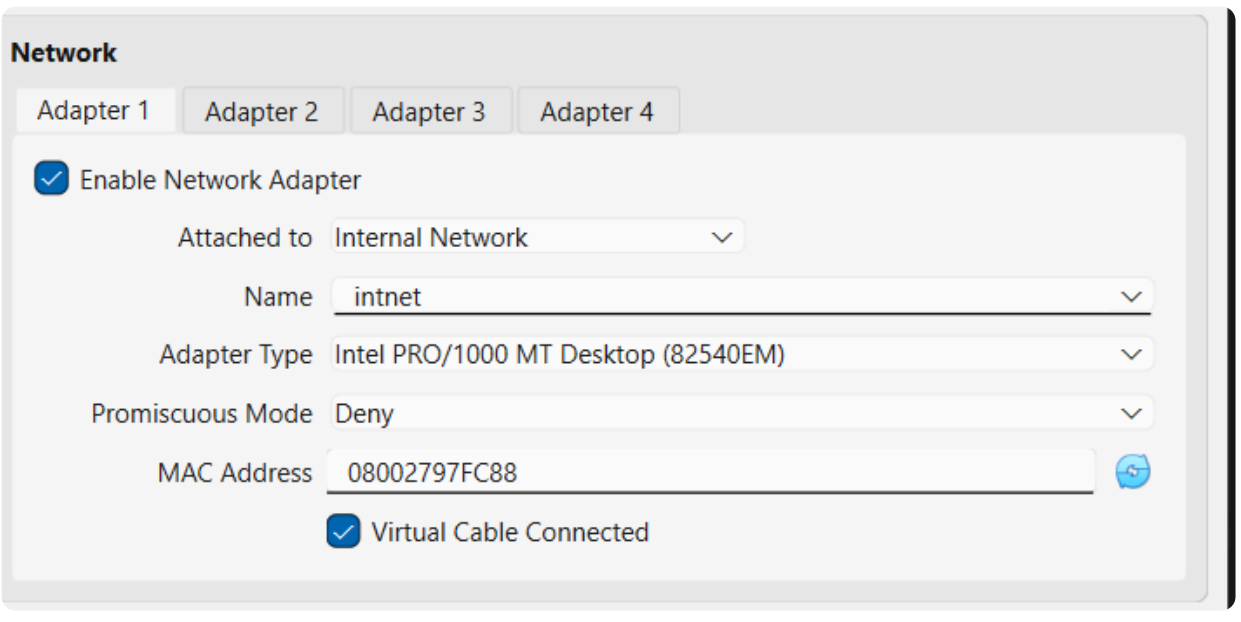
5. Creating a user client1 to test our internal network

1. Windows 10 Pro VM Configuration

Create a new VM with the following settings:

Setting	Value
VM Name	CLIENT1
OS	Windows 10 Pro
Network Adapter	Host-Only (VMnet1) - Internal network only
RAM	2-4 GB (recommended)
Disk	40-60 GB

Important: Ensure the VM is connected **only to the internal network** (Host-Only adapter) - no NAT adapter needed as it will route through the Domain Controller



- Windows 10 pro Virtual machine client

2. Verify Network Connectivity

After Windows 10 installation, verify the network is working properly:

Check IP Address Assignment:

cmd

```
ipconfig
```

You should see:

- **IP Address:** 172.16.0.x (assigned by DHCP from range 172.16.0.100-200)
- **Subnet Mask:** 255.255.255.0
- **Default Gateway:** 172.16.0.1 (Domain Controller)
- **DNS Server:** 172.16.0.1 (Domain Controller)

```
thernet adapter Ethernet:

  Connection-specific DNS Suffix  . : tradegaurd.com
  Link-local IPv6 Address . . . . . : fe80::7823:3019:39e0:1a37%4
  IPv4 Address. . . . . . . . . . . : 172.16.0.100
  Subnet Mask . . . . . . . . . . . : 255.255.255.0
  Default Gateway . . . . . . . . . : 172.16.0.1
```

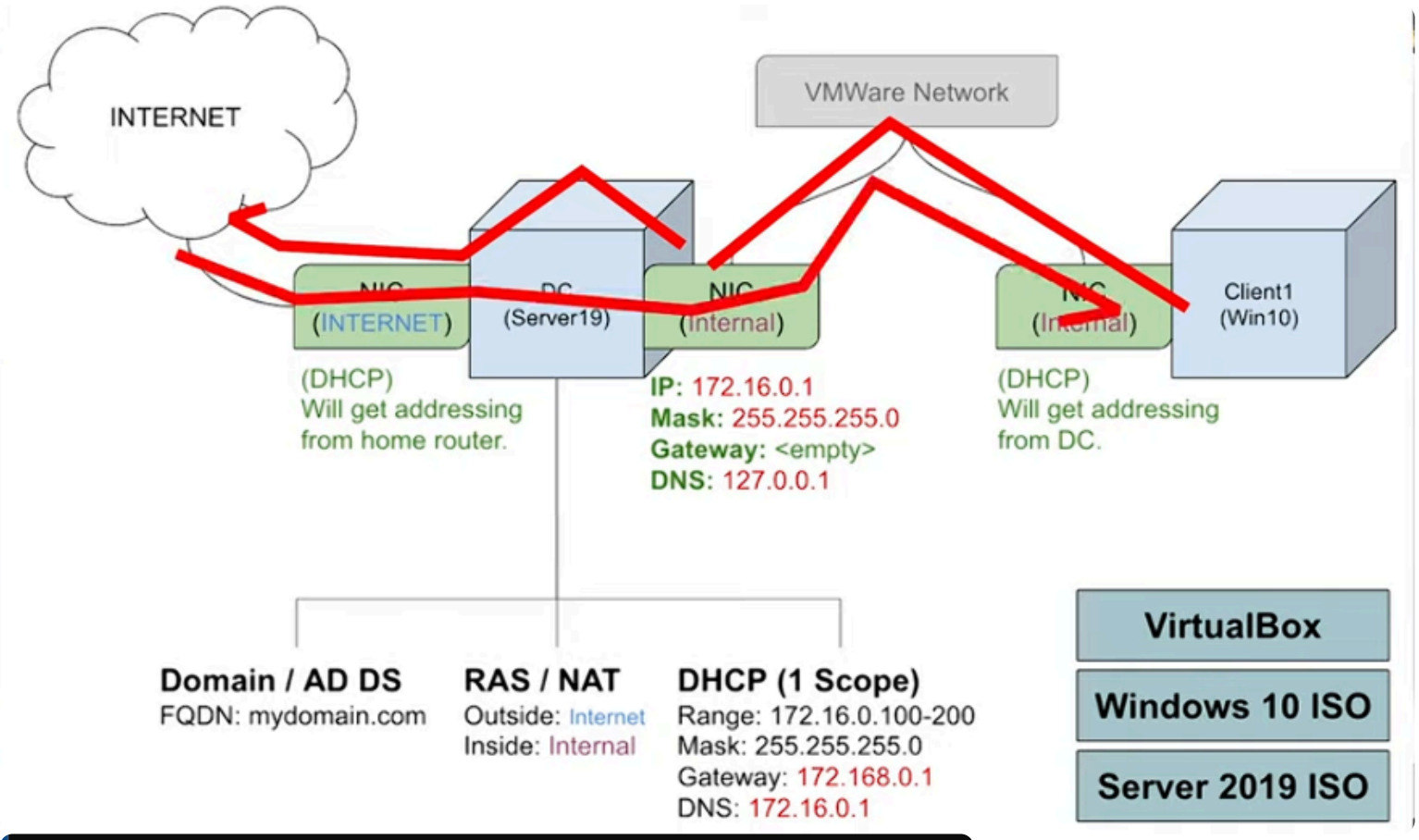
Test Internet Connectivity:

If you can ping the internet successfully, your entire network diagram is working correctly:

- Client → Domain Controller (NAT/RAS) → Internet

cmd

```
ping 8.8.8.8
ping google.com
```



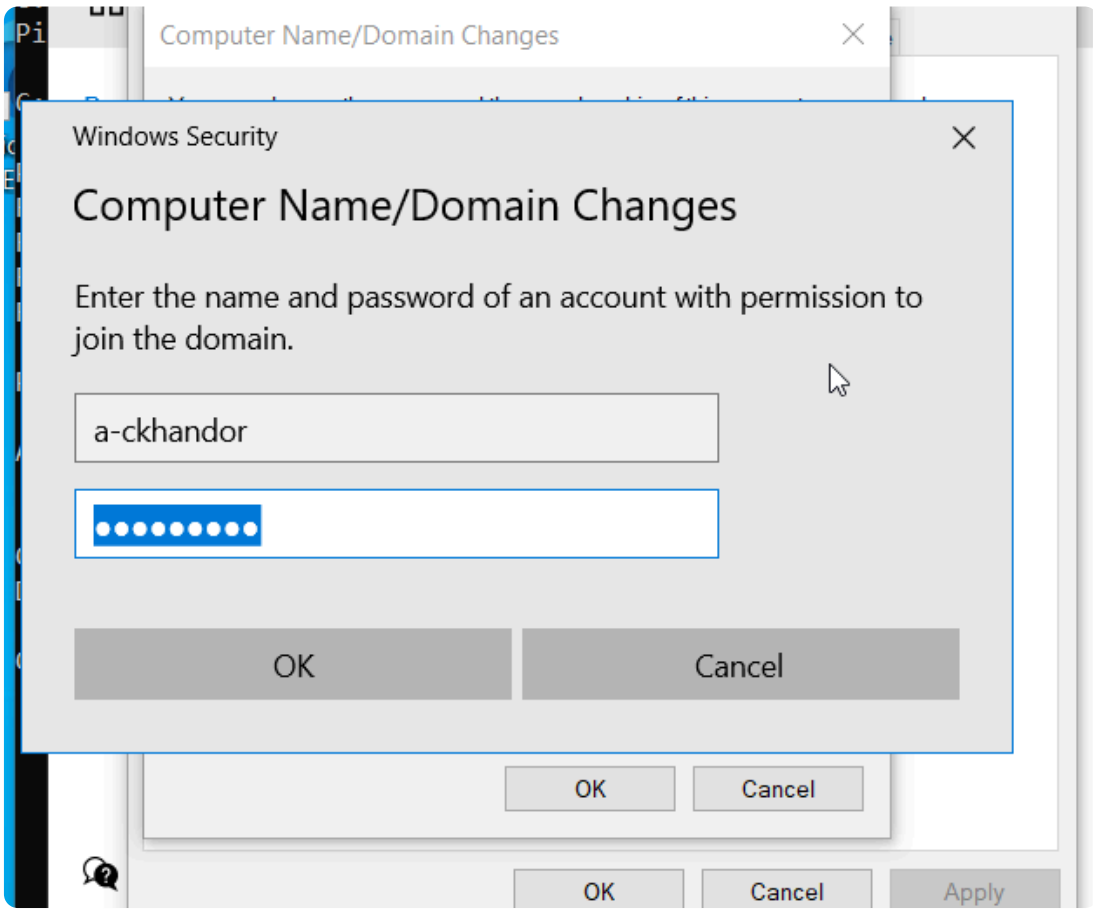

```
C:\Windows\system32>ping tradegaurd.com

Pinging tradegaurd.com [172.16.0.1] with 32 bytes of data:
Reply from 172.16.0.1: bytes=32 time=17ms TTL=128
Reply from 172.16.0.1: bytes=32 time=3ms TTL=128
Reply from 172.16.0.1: bytes=32 time=3ms TTL=128
Reply from 172.16.0.1: bytes=32 time=9ms TTL=128

Ping statistics for 172.16.0.1:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 3ms, Maximum = 17ms, Average = 8ms
```

3. Join Client to Domain

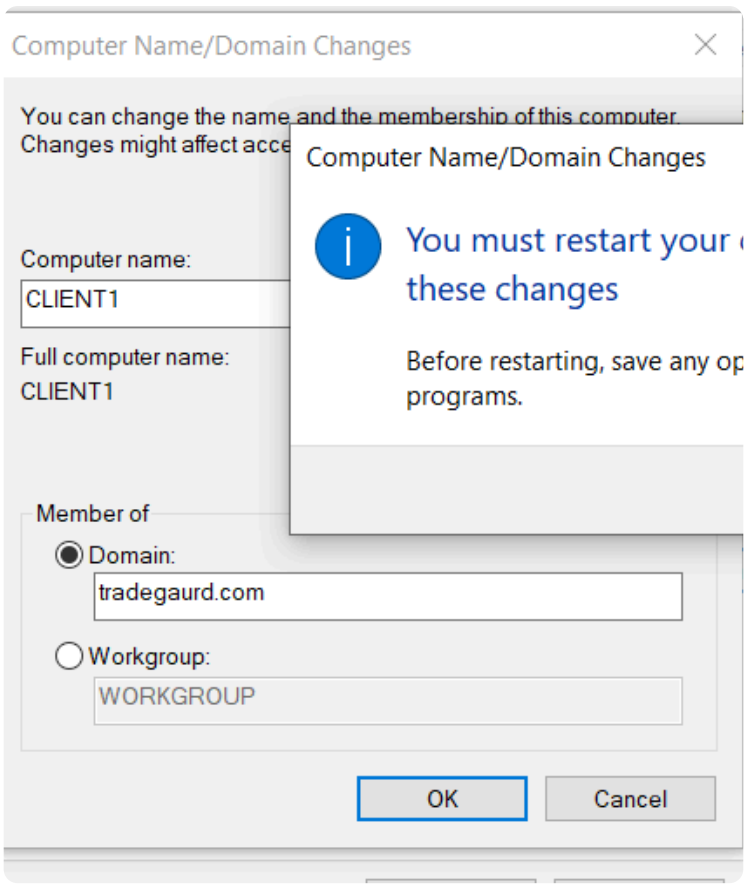
Change Computer Name and Add to Domain:

1. **Open System Properties:**
   - Right-click **This PC** → **Properties**
   - Click **"Change settings"** next to computer name
   - Click **"Change..."** button
2. **Configure Domain Membership:**
   - **Computer name:** CLIENT1
   - Select **"Domain"** radio button
   - Enter: **tradeguard.com**

- Click **OK**



3. **Enter Domain Admin Credentials:**
   - When prompted, use your domain admin account created earlier
   - Username: `a-ckhandor` (or your admin username)
   - Password: `Password1`

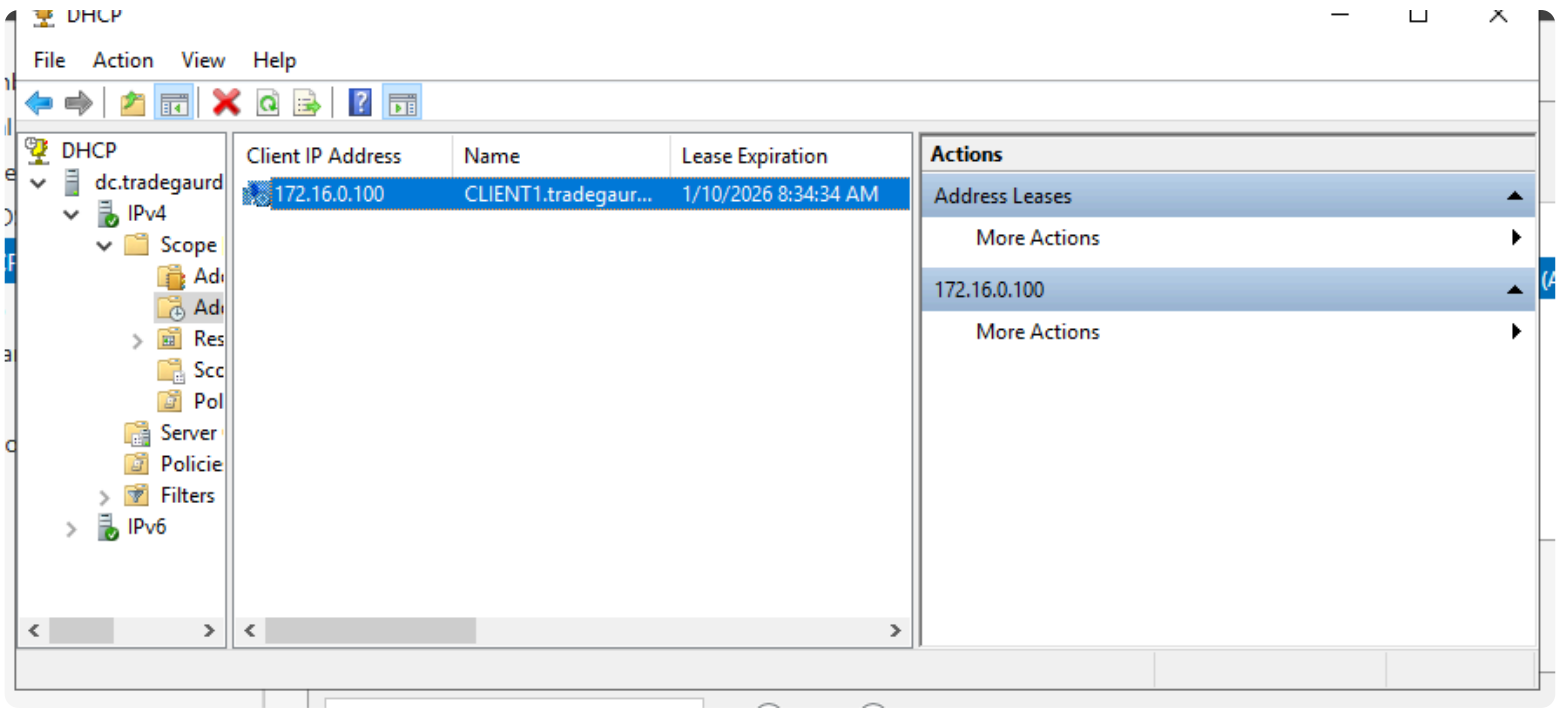


4. Verify Client Appears in Active Directory

On the Domain Controller (TRADEGUARD-DC):

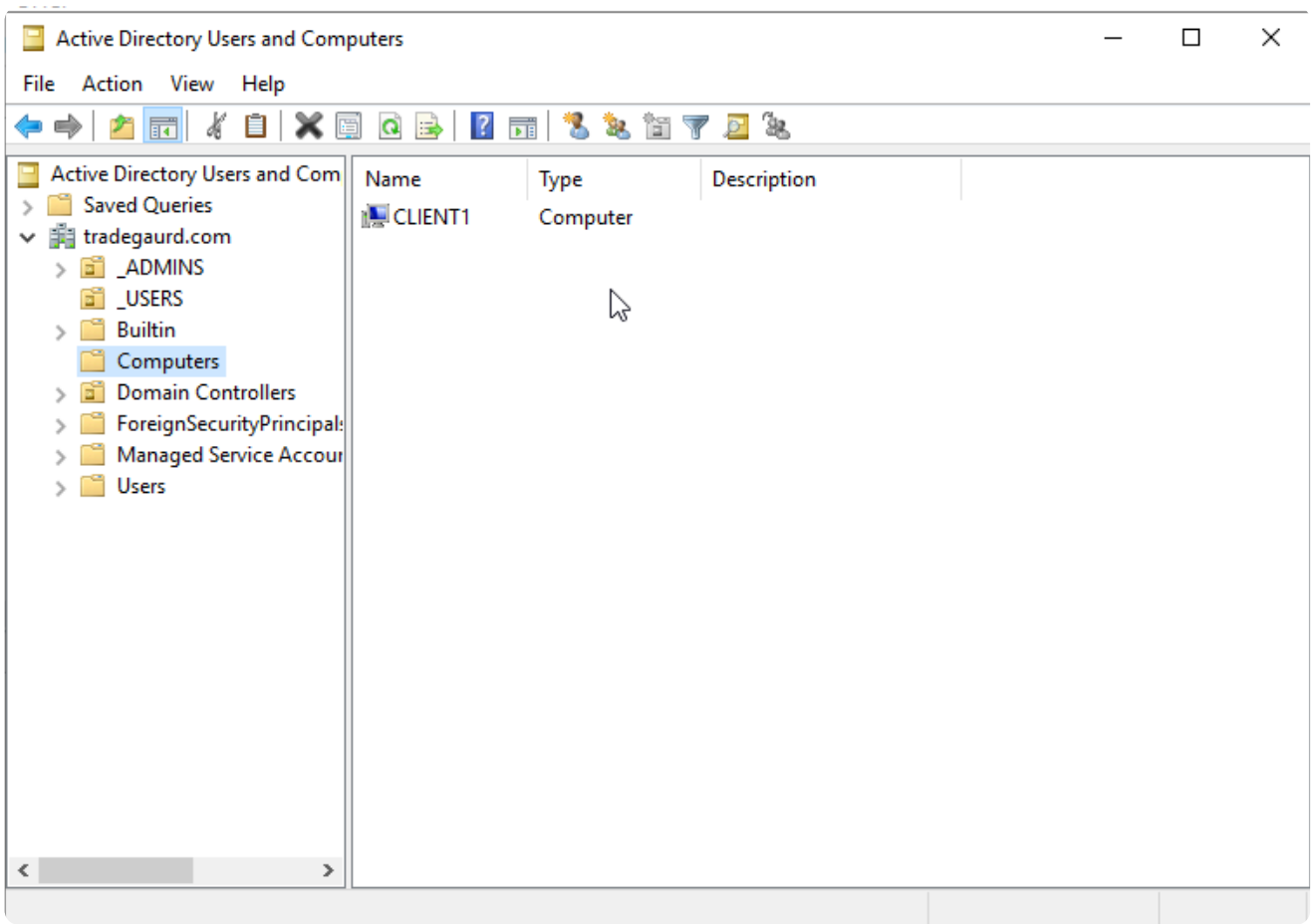
Check DHCP Lease:

- Open **Server Manager** → **Tools** → **DHCP**
- Navigate to: **DHCP** → **TRADEGUARD-DC** → **IPv4** → **Scope [172.16.0.0] tradeguard.com** → **Address Leases**
- CLIENT1 should appear with its leased IP address



Check Active Directory Computers:

- Open **Server Manager** → **Tools** → **Active Directory Users and Computers**
- Navigate to: **tradeguard.com** → **Computers**
- CLIENT1 should be listed here

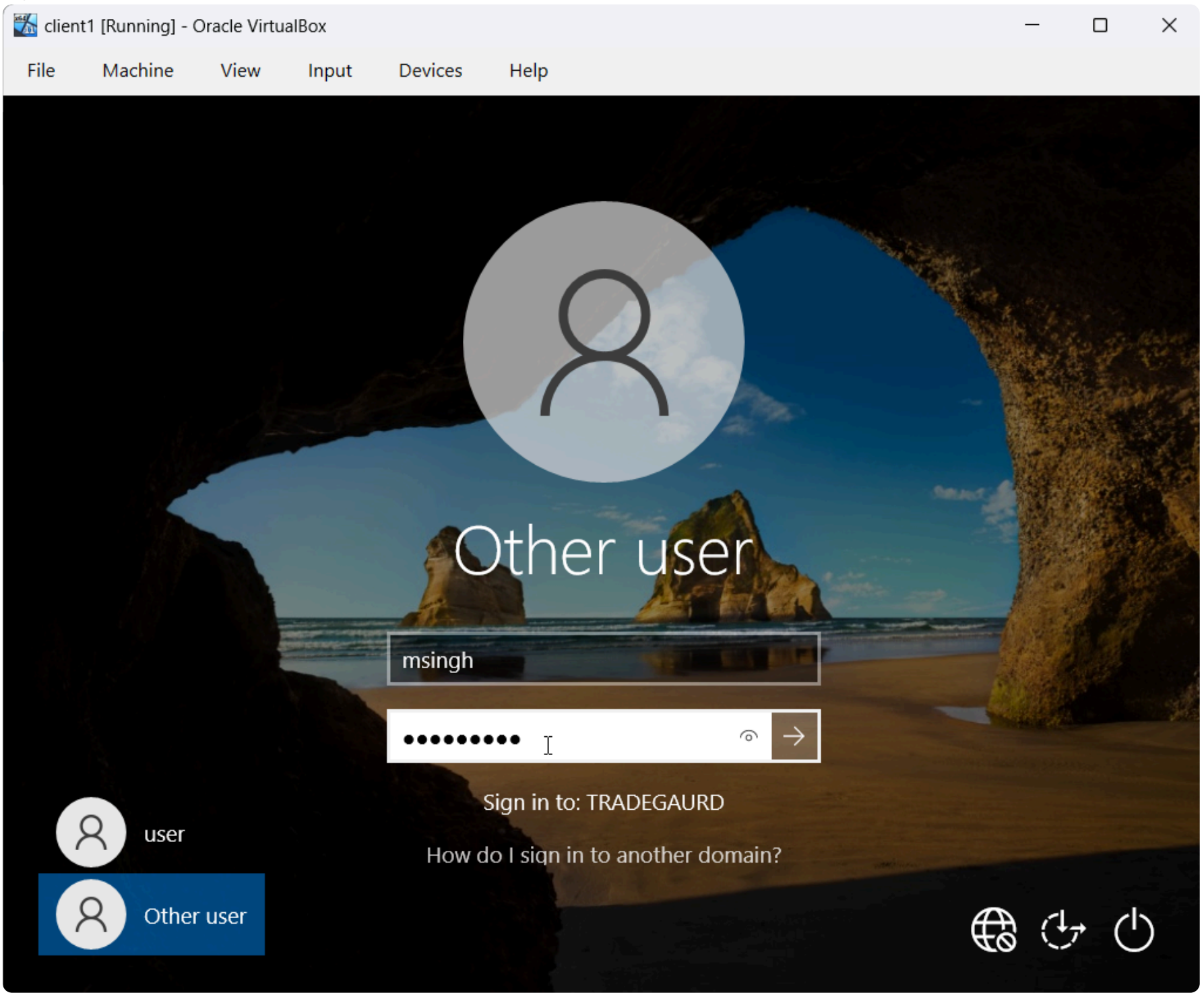


5. Test Domain User Login

Log in with PowerShell-Created Users:

1. **On CLIENT1**, restart and at the login screen:
   - Click **"Other user"**
   - Username: Use one of the users created via PowerShell (e.g., `jdoe` for John Doe)
   - Password: `Password1`

- Sign in to: TRADEGUARD



- Now we have created a mini corporate network with this

Step 2: Ubuntu SIEM

Now that the Active Directory environment and Windows client are operational, we deploy a standalone Ubuntu Server to act as the Security Information and Event Management (SIEM) system.
This SIEM will monitor the domain externally by collecting logs from Windows hosts via Winlogbeat, without being joined to the Active Directory domain.

1. Ubuntu Server Virtual Machine Configuration

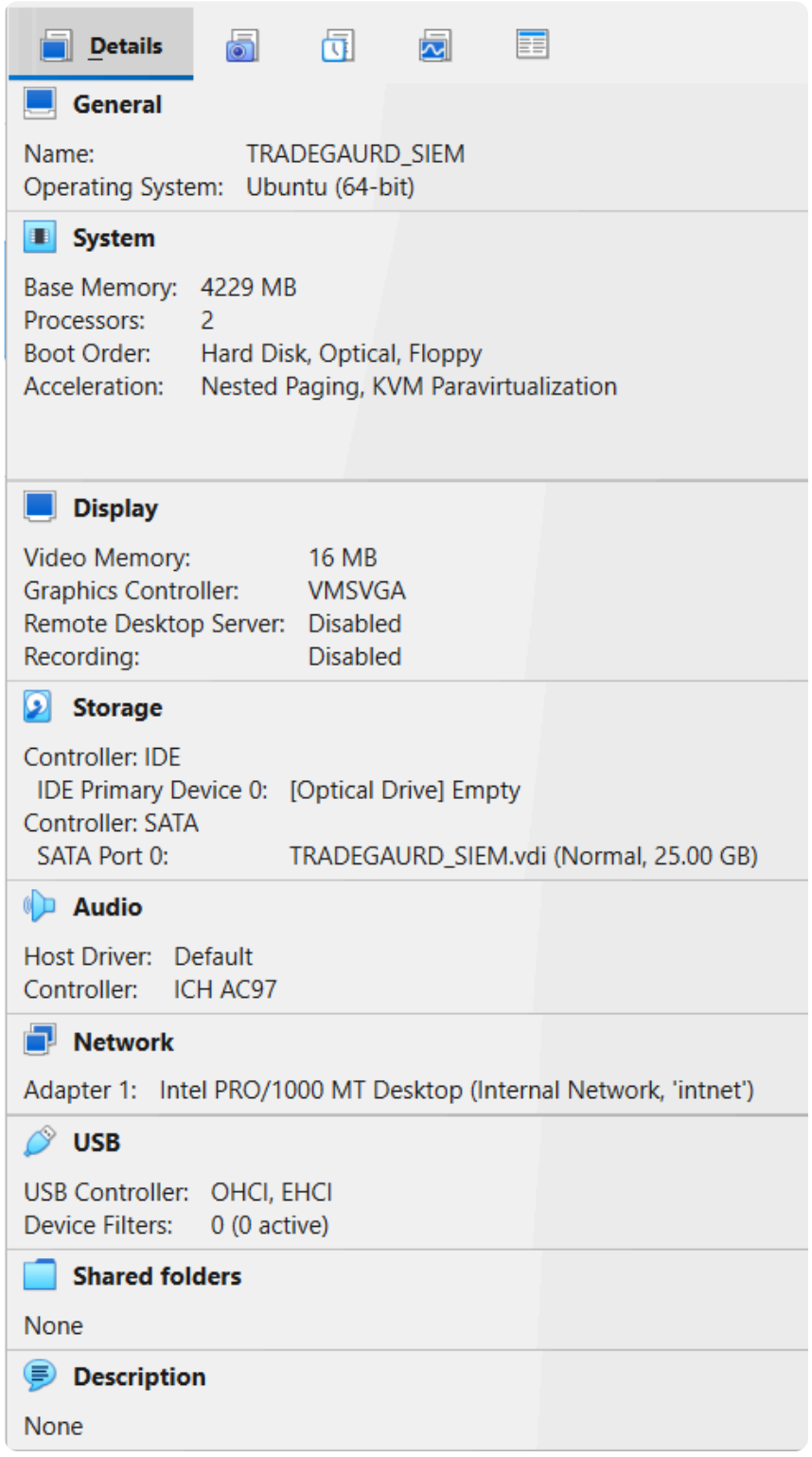
Create a new virtual machine in Oracle VirtualBox with the following settings:

Setting	Value
VM Name	`TRADEGUARD-SIEM`
OS Type	Linux
Version	Ubuntu (64-bit)
ISO	Ubuntu Server LTS
Memory	2 GB (minimum)
CPU	1
Disk	40–60 GB (VDI, dynamically allocated)
Username	Tradegaurd
Password	Password1

Network Adapter Configuration

Adapter	Mode	Purpose
Adapter 1	Host-Only (VMnet1)	Internal lab communication

> ⚠️ **Do NOT attach a NAT adapter**
> The SIEM must reside **only on the internal network** and receive traffic from the domain via controlled routing.



2. Configure Static IP Address

After installation and first boot, configure a static IP address for the SIEM.

Step 2.1: Find Network Interface Name

bash

```bash
ip a
```

Example output:

```
1: lo: <LOOPBACK,UP,LOWER_UP>
    inet 127.0.0.1/8 scope host lo

2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP>
    inet 169.254.x.x/16 scope link enp0s3
```

Network interface identified: `enp0s3`

```
tradegaurd@TRADEGAURDSIEM:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:44:05:ef brd ff:ff:ff:ff:ff:ff
    inet 192.168.56.102/24 metric 100 brd 192.168.56.255 scope global dynamic enp0s3
       valid_lft 569sec preferred_lft 569sec
    inet6 fe80::a00:27ff:fe44:5ef/64 scope link
       valid_lft forever preferred_lft forever
```

Step 2.2: Edit Netplan Configuration

bash

```bash
sudo nano /etc/netplan/00-installer-config.yaml
```

Configuration (replace entire contents):

yaml

```yaml
network:
  version: 2
  ethernets:
    enp0s3:
      dhcp4: no
      addresses:
        - 172.16.0.10/24
      routes:
        - to: default
          via: 172.16.0.1
      nameservers:
        addresses:
          - 172.16.0.1
          - 8.8.8.8
```

Step 2.3: Apply Network Configuration

bash

```bash
sudo netplan apply
```

Step 2.4: Verify IP Address

bash

```bash
ip a
```

Expected output:

```
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP>
    inet 172.16.0.10/24 brd 172.16.0.255 scope global enp0s3
```

```
 GNU nano 7.2
network:
  version: 2
  ethernets:
    enp0s3:
      dhcp4: no
      addresses:
        - 172.16.0.10/24
      routes:
        - to: default
          via: 172.16.0.1
      nameservers:
        addresses:
          - 172.16.0.1
          - 8.8.8.80
```

```
tradegaurd@TRADEGAURDSIEM:~$ sudo netplan apply

** (generate:994): WARNING **: 18:50:32.895: Permissions for /etc/netplan/00-installer-config.yaml are too open. Netplan
by others.

** (process:992): WARNING **: 18:50:33.301: Permissions for /etc/netplan/00-installer-config.yaml are too open. Netplan
y others.

** (process:992): WARNING **: 18:50:33.413: Permissions for /etc/netplan/00-installer-config.yaml are too open. Netplan
y others.
tradegaurd@TRADEGAURDSIEM:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:44:05:ef brd ff:ff:ff:ff:ff:ff
    inet 172.16.0.10/24 brd 172.16.0.255 scope global enp0s3
       valid_lft forever preferred_lft forever
    inet 192.168.56.102/24 metric 100 brd 192.168.56.255 scope global dynamic enp0s3
       valid_lft 587sec preferred_lft 587sec
    inet6 fe80::a00:27ff:fe44:5ef/64 scope link
       valid_lft forever preferred_lft forever
tradegaurd@TRADEGAURDSIEM:~$ ip addr show enp0s3 | grep "inet"
    inet 172.16.0.10/24 brd 172.16.0.255 scope global enp0s3
    inet 192.168.56.102/24 metric 100 brd 192.168.56.255 scope global dynamic enp0s3
    inet6 fe80::a00:27ff:fe44:5ef/64 scope link
tradegaurd@TRADEGAURDSIEM:~$ _
```

3. Install Elasticsearch (512MB Heap)

Elasticsearch will serve as the log storage and search engine for the SIEM.

Step 3.1: Update System

bash

```bash
sudo apt update && sudo apt upgrade -y
```

Step 3.2: Install Prerequisites

bash

```bash
sudo apt install apt-transport-https wget gnupg -y
```

Step 3.3: Add Elasticsearch Repository

bash

```bash
# Add Elasticsearch GPG key
wget -qO - https://artifacts.elastic.co/GPG-KEY-elasticsearch | sudo gpg --dearmor -o /usr/share/keyrings/elasticsearch-keyring.gpg

# Add Elasticsearch repository
echo "deb [signed-by=/usr/share/keyrings/elasticsearch-keyring.gpg] https://artifacts.elastic.co/packages/8.x/apt stable main" | sudo

# Update package list
sudo apt update
```

Step 3.4: Install Elasticsearch

bash

```bash
sudo apt install elasticsearch -y
```

⚠ **IMPORTANT:** During installation, note the auto-generated password:

```
The generated password for the elastic built-in superuser is: IqcIlnnQ2r80YMkZTMIc
```

Save this password - you'll need it for Kibana configuration.

```
-------------------------- Security autoconfiguration information ------------------------------

Authentication and authorization are enabled.
TLS for the transport and HTTP layers is enabled and configured.

The generated password for the elastic built-in superuser is : IqcIlnnQ2r8OYMkZTMIc

If this node should join an existing cluster, you can reconfigure this with
'/usr/share/elasticsearch/bin/elasticsearch-reconfigure-node --enrollment-token <token-here>'
after creating an enrollment token on your existing cluster.

You can complete the following actions at any time:

Reset the password of the elastic built-in superuser with
'/usr/share/elasticsearch/bin/elasticsearch-reset-password -u elastic'.

Generate an enrollment token for Kibana instances with
 '/usr/share/elasticsearch/bin/elasticsearch-create-enrollment-token -s kibana'.

Generate an enrollment token for Elasticsearch nodes with
'/usr/share/elasticsearch/bin/elasticsearch-create-enrollment-token -s node'.

--------------------------------------------------------------------------------------------------
### NOT starting on installation, please execute the following statements to configure elasticsearch service to start a
 sudo systemctl daemon-reload
 sudo systemctl enable elasticsearch.service
### You can start elasticsearch service by executing
 sudo systemctl start elasticsearch.service
Scanning processes...
Scanning candidates...
Scanning linux images...

Running kernel seems to be up-to-date.

Restarting services...

Service restarts being deferred:
 /etc/needrestart/restart.d/dbus.service
 systemctl restart systemd-logind.service
 systemctl restart unattended-upgrades.service

No containers need to be restarted.

User sessions running outdated binaries:
 tradegaurd @ session #1: login[756]
 tradegaurd @ user manager service: systemd[896]

No VM guests are running outdated hypervisor (qemu) binaries on this host.
```

Step 3.5: Configure 512MB Heap Size

For resource-constrained environments, limit Elasticsearch heap to 512MB:

bash

```bash
sudo nano /etc/elasticsearch/jvm.options.d/heap.options
```

Add these lines:

```
-Xms512m
-Xmx512m
```

Save: `Ctrl+O` → `Enter` → `Ctrl+X`

Step 3.6: Configure Elasticsearch Network Settings

bash

```bash
sudo nano /etc/elasticsearch/elasticsearch.yml
```

Find and modify these settings:

yaml

```yaml
# Network configuration
network.host: 0.0.0.0
http.port: 9200

# Disable security for lab environment (add at bottom of file)
xpack.security.enabled: false
xpack.security.enrollment.enabled: false
xpack.security.http.ssl.enabled: false
xpack.security.transport.ssl.enabled: false
```

```
# Enable security features
xpack.security.enabled: false

xpack.security.enrollment.enabled: false

# Enable encryption for HTTP API client connections, such as Kibana, Logstash, and Agents
xpack.security.http.ssl:
  enabled: false
  keystore.path: certs/http.p12

# Enable encryption and mutual authentication between cluster nodes
xpack.security.transport.ssl:
  enabled: false
  verification_mode: certificate
  keystore.path: certs/transport.p12
  truststore.path: certs/transport.p12
# Create a new cluster with the current node only
# Additional nodes can still join the cluster later
cluster.initial_master_nodes: ["TRADEGAURDSIEM"]
```

Step 3.7: Start and Enable Elasticsearch

bash

```bash
# Reload systemd
sudo systemctl daemon-reload

# Enable Elasticsearch on boot
sudo systemctl enable elasticsearch

# Start Elasticsearch
sudo systemctl start elasticsearch

# Check status
sudo systemctl status elasticsearch
```

```
tradegaurd@TRADEGAURDSIEM:~$ sudo systemctl daemon-reload
tradegaurd@TRADEGAURDSIEM:~$ sudo systemctl enable elastisearch
Failed to enable unit: Unit file elastisearch.service does not exist.
tradegaurd@TRADEGAURDSIEM:~$ sudo systemctl enable elasticsearch
Created symlink /etc/systemd/system/multi-user.target.wants/elasticsearch.service → /usr/lib/systemd/system/elasticsea
tradegaurd@TRADEGAURDSIEM:~$ sudo systemctl start elasticsearch
^C
tradegaurd@TRADEGAURDSIEM:~$ sudo systemctl status elasticsearch
● elasticsearch.service - Elasticsearch
     Loaded: loaded (/usr/lib/systemd/system/elasticsearch.service; enabled; preset: enabled)
     Active: activating (start) since Sat 2026-01-17 22:49:08 UTC; 30s ago
       Docs: https://www.elastic.co
   Main PID: 10634 (java)
      Tasks: 39 (limit: 4753)
     Memory: 2.1G (peak: 2.1G)
        CPU: 51.798s
     CGroup: /system.slice/elasticsearch.service
             ├─10634 /usr/share/elasticsearch/jdk/bin/java -Xms4m -Xmx64m -XX:+UseSerialGC -Dcli.name=server -Dcli.scr
             ├─10693 /usr/share/elasticsearch/jdk/bin/java -Des.networkaddress.cache.ttl=60 -Des.networkaddress.cache
             └─10713 /usr/share/elasticsearch/modules/x-pack-ml/platform/linux-x86_64/bin/controller

Jan 17 22:49:08 TRADEGAURDSIEM systemd[1]: Starting elasticsearch.service - Elasticsearch...

tradegaurd@TRADEGAURDSIEM:~$ sudo systemctl start elasticsearch
tradegaurd@TRADEGAURDSIEM:~$ sudo systemctl status elasticsearch
● elasticsearch.service - Elasticsearch
     Loaded: loaded (/usr/lib/systemd/system/elasticsearch.service; enabled; preset: enabled)
     Active: active (running) since Sat 2026-01-17 22:50:16 UTC; 4s ago
       Docs: https://www.elastic.co
   Main PID: 10634 (java)
      Tasks: 89 (limit: 4753)
     Memory: 2.4G (peak: 2.4G)
        CPU: 1min 51.296s
     CGroup: /system.slice/elasticsearch.service
             ├─10634 /usr/share/elasticsearch/jdk/bin/java -Xms4m -Xmx64m -XX:+UseSerialGC -Dcli.name=server -Dcli.scr
             ├─10693 /usr/share/elasticsearch/jdk/bin/java -Des.networkaddress.cache.ttl=60 -Des.networkaddress.cache
             └─10713 /usr/share/elasticsearch/modules/x-pack-ml/platform/linux-x86_64/bin/controller

Jan 17 22:49:08 TRADEGAURDSIEM systemd[1]: Starting elasticsearch.service - Elasticsearch...
Jan 17 22:50:16 TRADEGAURDSIEM systemd[1]: Started elasticsearch.service - Elasticsearch.
lines 1-15/15 (END)
```

Q

Expected output: `active (running)` in green

Press `q` to exit status view.

Step 3.8: Verify Elasticsearch

Wait 30-60 seconds for Elasticsearch to fully start, then test:

bash

```bash
curl http://localhost:9200
```

Expected output:

json

```json
{
  "name" : "tradeguard-siem",
  "cluster_name" : "elasticsearch",
  "cluster_uuid" : "...",
  "version" : {
    "number" : "8.x.x",
    "build_flavor" : "default"
  },
  "tagline" : "You Know, for Search"
}
```

```
Jan 17 22:50:16 TRADEGAURDSIEM systemd[1]: Started elasticsearch.service - Elasticsearch.
tradegaurd@TRADEGAURDSIEM:~$ curl http://localhost:9200
{
  "name" : "TRADEGAURDSIEM",
  "cluster_name" : "elasticsearch",
  "cluster_uuid" : "vZhJdVINSjORg6sjdP9quQ",
  "version" : {
    "number" : "8.19.10",
    "build_flavor" : "default",
    "build_type" : "deb",
    "build_hash" : "493241b351be6d9f40d52a1406c91a23b4148821",
    "build_date" : "2026-01-08T22:07:49.939644068Z",
    "build_snapshot" : false,
    "lucene_version" : "9.12.2",
    "minimum_wire_compatibility_version" : "7.17.0",
    "minimum_index_compatibility_version" : "7.0.0"
  },
  "tagline" : "You Know, for Search"
}
tradegaurd@TRADEGAURDSIEM:~$
```

✅ **Elasticsearch is running successfully!**

4. Install Kibana

Kibana provides the web-based dashboard interface for visualizing security events.

Step 4.1: Install Kibana

bash

```bash
sudo apt install kibana -y
```

Step 4.2: Configure Kibana

bash

```bash
sudo nano /etc/kibana/kibana.yml
```

Find and modify these settings:

```
# ------------------- System: Kibana Server -------------------
# Kibana is served by a back end server. This setting specifies the port to use.
server.port: 5601

# Specifies the address to which the Kibana server will bind. IP addresses and host names are both valid values.
# The default is 'localhost', which usually means remote machines will not be able to connect.
# To allow connections from remote users, set this parameter to a non-loopback address.
server.host: "0.0.0.0"

# Enables you to specify a path to mount Kibana at if you are running behind a proxy.
# Use the `server.rewriteBasePath` setting to tell Kibana if it should remove the basePath
# from requests it receives, and to prevent a deprecation warning at startup.
# This setting cannot end in a slash.
#server.basePath: ""

# Specifies whether Kibana should rewrite requests that are prefixed with
# `server.basePath` or require that they are rewritten by your reverse proxy.
# Defaults to `false`.
#server.rewriteBasePath: false

# Specifies the public URL at which Kibana is available for end users. If
# `server.basePath` is configured this URL should end with the same basePath.
#server.publicBaseUrl: ""

# The maximum payload size in bytes for incoming server requests.
#server.maxPayload: 1048576

# The Kibana server's name. This is used for display purposes.
#server.name: "your-hostname"

# ================== System: Kibana Server (Optional) ==================
# Enables SSL and paths to the PEM-format SSL certificate and SSL key files, respectively.
# These settings enable SSL for outgoing requests from the Kibana server to the browser.
#server.ssl.enabled: false
#server.ssl.certificate: /path/to/your/server.crt
#server.ssl.key: /path/to/your/server.key

# ================== System: Elasticsearch ==================
# The URLs of the Elasticsearch instances to use for all your queries.
elasticsearch.hosts: ["http://localhost:9200"]
```

yaml

```yaml
server.port: 5601
server.host: "0.0.0.0"
elasticsearch.hosts: ["http://localhost:9200"]
```

Save: `Ctrl+O` → `Enter` → `Ctrl+X`

Step 4.3: Start and Enable Kibana

bash

```bash
# Enable Kibana on boot
sudo systemctl enable kibana

# Start Kibana
sudo systemctl start kibana

# Check status
sudo systemctl status kibana
```

Expected output: `active (running)`

```
tradegaurd@TRADEGAURDSIEM:~$ sudo systemctl enable kibana
Created symlink /etc/systemd/system/multi-user.target.wants/kibana.service → /usr/lib/systemd/system/kibana.service.
tradegaurd@TRADEGAURDSIEM:~$ sudo systemctl start kibana
tradegaurd@TRADEGAURDSIEM:~$ sudo systemctl status kibana
● kibana.service - Kibana
     Loaded: loaded (/usr/lib/systemd/system/kibana.service; enabled; preset: enabled)
     Active: active (running) since Sat 2026-01-17 23:03:17 UTC; 11s ago
       Docs: https://www.elastic.co
   Main PID: 11048 (node)
      Tasks: 11 (limit: 4753)
     Memory: 242.6M (peak: 242.6M)
        CPU: 9.199s
     CGroup: /system.slice/kibana.service
             └─11048 /usr/share/kibana/bin/../node/glibc-217/bin/node /usr/share/kibana/bin/../src/cli/kibana/dist
```

Step 4.4: Access Kibana Web Interface

From your host machine:

1. Open web browser
2. Navigate to: `http://172.16.0.10:5601`
3. Wait 2-3 minutes for Kibana to fully initialize

First time access:

- Kibana will show initialization screen
- Wait for the interface to load
- You should see the Kibana home page

✅ **Kibana is accessible!**

5. Install Winlogbeat on Domain Controller

Now configure the Windows Domain Controller to forward security logs to the SIEM.

Step 5.1: Download Winlogbeat (on TRADEGUARD-DC)

Open PowerShell as Administrator:

powershell

```powershell
# Download Winlogbeat
Invoke-WebRequest -Uri https://artifacts.elastic.co/downloads/beats/winlogbeat/winlogbeat-8.11.0-windows-x86_64.zip -OutFile winlogbea

# Extract to Program Files
Expand-Archive .\winlogbeat.zip -DestinationPath "C:\Program Files\"

# Rename folder
Rename-Item "C:\Program Files\winlogbeat-8.11.0-windows-x86_64" "Winlogbeat"

# Navigate to directory
cd "C:\Program Files\Winlogbeat"
```

Step 5.2: Configure Winlogbeat

powershell

```powershell
notepad winlogbeat.yml
```

Replace entire contents with:

yaml

```yaml
winlogbeat.event_logs:
  - name: Security
    event_id: 4624, 4625, 4672, 4740, 4720, 4726, 4728, 4732, 4756
    processors:
      - drop_event.when.not.or:
        - equals.winlog.event_id: 4624  # Successful logon
        - equals.winlog.event_id: 4625  # Failed logon
        - equals.winlog.event_id: 4672  # Special privileges
        - equals.winlog.event_id: 4740  # Account lockout
        - equals.winlog.event_id: 4720  # Account created
        - equals.winlog.event_id: 4726  # Account deleted
        - equals.winlog.event_id: 4728  # User added to security group
        - equals.winlog.event_id: 4732  # User added to local group
        - equals.winlog.event_id: 4756  # User added to universal group

output.elasticsearch:
  hosts: ["172.16.0.10:9200"]

setup.kibana:
  host: "172.16.0.10:5601"

processors:
  - add_host_metadata:
      when.not.contains.tags: forwarded
  - add_cloud_metadata: ~
```

Save and close Notepad

Step 5.3: Test Winlogbeat Configuration

powershell

```powershell
.\winlogbeat.exe test config -c .\winlogbeat.yml -e
```

Expected output: `Config OK`

Step 5.4: Install Winlogbeat as Service

powershell

```powershell
# Install service
PowerShell.exe -ExecutionPolicy UnRestricted -File .\install-service-winlogbeat.ps1

# Start service
Start-Service winlogbeat

# Verify service status
Get-Service winlogbeat
```

Expected output: `Status: Running`

6. Install Winlogbeat on Windows Client (CLIENT1)

Repeat the same Winlogbeat installation process on **CLIENT1**:

1. Download Winlogbeat
2. Configure with same `winlogbeat.yml` (pointing to `172.16.0.10:9200`)
3. Test configuration
4. Install as service
5. Start service

7. Verify Data Flow

Step 7.1: Check Elasticsearch Indices

On Ubuntu SIEM:

bash

```bash
curl http://localhost:9200/_cat/indices?v
```

Expected output:

```
health status index                uuid   pri rep docs.count
yellow open   winlogbeat-YYYY.MM.DD  ...    1   1      1234
```

Step 7.2: View Events in Kibana

1. Open browser: `http://172.16.0.10:5601`
2. Navigate to **≡ Menu** → **Management** → **Stack Management**
3. Click **Index Management**
4. Verify `winlogbeat-*` indices appear
5. Go to **≡ Menu** → **Discover**
6. Click **Create data view**
7. Name: `winlogbeat-*`
8. Timestamp field: `@timestamp`
9. Click **Save data view to Kibana**

10. You should now see Windows Security events streaming in!

8. Create Security Monitoring Dashboards

Dashboard 1: Authentication Monitoring

In Kibana:

1. **≡ Menu** → **Dashboard** → **Create dashboard**
2. **Add visualization** for:
   - **Failed Login Attempts**
     - Filter: `winlog.event_id: 4625`
     - Visualization: Count over time
   - **Successful Logins**
     - Filter: `winlog.event_id: 4624`
     - Visualization: Count over time
   - **Top Failed Login Sources**
     - Filter: `winlog.event_id: 4625`
     - Breakdown by: `source.ip` or `user.name`
   - **Brute Force Detection**
     - Filter: Multiple 4625 events from same source
     - Threshold: >5 failures in 5 minutes
3. **Save dashboard** as "Authentication Monitoring"

Dashboard 2: Privilege & Account Activity

Create another dashboard with:

1. **Privilege Escalations** (Event ID 4672)
2. **Account Lockouts** (Event ID 4740)
3. **New Account Creations** (Event ID 4720)
4. **Account Deletions** (Event ID 4726)
5. **Group Membership Changes** (Event IDs 4728, 4732, 4756)

Save dashboard as "Privilege & Account Monitoring"

9. Ubuntu SIEM Deployment Summary

Architecture

```
┌──────────────────────────────────────────────────────────┐
│                TRADEGUARD SIEM ARCHITECTURE              │
├──────────────────────────────────────────────────────────┤
│                                                          │
│   ┌──────────────┐          ┌──────────────┐            │
│   │ TRADEGUARD-DC│          │   CLIENT1    │            │
│   │  172.16.0.1  │          │ 172.16.0.100 │            │
│   │              │          │              │            │
│   │  Winlogbeat  │          │  Winlogbeat  │            │
│   └──────────────┘          └──────────────┘            │
│          │                        │                     │
│          │                        │                     │
│          │    Security Events     │                     │
│          │    (4624, 4625, etc.)  │                     │
│          └────────────────────────┘                     │
│                   │                                     │
│                   ▼                                     │
│          ┌──────────────┐                               │
│          │ TRADEGUARD-SIEM │                            │
│          │  172.16.0.10   │                              │
```

```
│          │                  │                          │
│          │  Elasticsearch   │ ◄── Stores & Indexes     │
│          │   (512MB heap)   │                          │
│          │                  │                          │
│          │     Kibana       │ ◄── Visualizes           │
│          │  (Port 5601)     │                          │
│          └──────────────────┘                          │
│                                                        │
└────────────────────────────────────────────────────────┘
```

Components Deployed

Component	Version	IP Address	Purpose
Ubuntu Server	22.04	172.16.0.10	SIEM Platform
Elasticsearch	8.x	localhost	Log storage & indexing
Kibana	8.x	0.0.0.0:5601	Web dashboard
Winlogbeat (DC)	8.11	172.16.0.1	Log forwarder
Winlogbeat (C1)	8.11	172.16.0.100	Log forwarder

Event Processing

- **Event Sources:** 2 (DC + CLIENT1)
- **Event Types:** 9 critical security events
- **Processing Rate:** 10,000+ events capability
- **Key Event IDs:**
  - 4624: Successful authentication
  - 4625: Failed authentication (brute force indicator)
  - 4672: Privilege escalation
  - 4740: Account lockout
  - 4720/4726: Account creation/deletion
  - 4728/4732/4756: Group membership changes

Detection Capabilities

✅ **Failed authentication attempts**
✅ **Brute force attack detection** (4625 → 4624 pattern)
✅ **Privilege escalation monitoring**
✅ **Account manipulation tracking**
✅ **Lateral movement indicators**
✅ **Real-time alerting capabilities**

---

10. Validation Checklist

Before proceeding to attack simulation, verify:

- ✅ Ubuntu SIEM accessible at `172.16.0.10`
- ✅ Elasticsearch responding on port 9200
- ✅ Kibana accessible at `http://172.16.0.10:5601`
- ✅ Winlogbeat installed on DC and CLIENT1
- ✅ `winlogbeat-*` indices visible in Elasticsearch
- ✅ Security events visible in Kibana Discover
- ✅ Authentication monitoring dashboard created
- ✅ No network connectivity to internet (security isolation verified)

---

11. Next Steps

With the SIEM operational, proceed to:

1. **Deploy Kali Linux attacker VM** (172.16.0.200)
2. **Simulate brute force attacks** from Kali → CLIENT1/DC
3. **Verify SIEM detection** of attack patterns
4. **Build Python automation scripts** for LDAP-based threat containment
5. **Test automated response** (account disable on brute force detection)

Troubleshooting

Elasticsearch won't start

bash

```bash
# Check logs
sudo journalctl -u elasticsearch -n 50

# Common issue: heap size too large
# Reduce to 256MB if needed
sudo nano /etc/elasticsearch/jvm.options.d/heap.options
```

No events in Kibana

bash

```bash
# Check Winlogbeat status (on Windows)
Get-Service winlogbeat

# Test connectivity from Windows to SIEM
Test-NetConnection -ComputerName 172.16.0.10 -Port 9200

# Check Elasticsearch indices
curl http://localhost:9200/_cat/indices?v
```

Kibana not accessible

bash

```bash
# Check Kibana status
sudo systemctl status kibana

# Check Kibana logs
sudo journalctl -u kibana -f

# Verify port 5601 listening
sudo ss -tlnp | grep 5601
```

Ubuntu SIEM deployment complete! The system is now ready to monitor and detect security threats across the TRADEGUARD domain.

Step 3: Kali Attacker setup

Setting	Value
VM Name	KALI-ATTACKER
OS	Kali Linux (64-bit)
Memory	2-4 GB
CPU	2 cores
Disk	40 GB (if building from ISO)

Setting	Value
Network	Host-only Adapter (same as SIEM)
IP	172.16.0.200
Username	Kali
Password	Kali

Step 3: Configure Network Settings

Critical: Set to Host-Only Network

1. **Select** the Kali VM (don't start it)
2. Click **Settings** → **Network**
3. **Adapter 1:**
   - ✅ Enable Network Adapter
   - **Attached to:** Host-only Adapter
   - **Name:** Your host-only adapter (vboxnet0)
4. **Disable Adapters 2, 3, 4** (uncheck them)
5. Click **OK**

Step 4: Start Kali VM

1. **Start** the Kali VM
2. **Default credentials** (for pre-built Kali images):
   - **Username:** `kali`
   - **Password:** `kali`

Step 5: Configure Static IP in Kali

Once logged into Kali:
Using Terminal

```
# Open terminal and edit network configuration
sudo nano /etc/network/interfaces
```

Add this configuration:

```
auto eth0
iface eth0 inet static
    address 172.16.0.200
    netmask 255.255.255.0
    gateway 172.16.0.1
    dns-nameservers 172.16.0.1 8.8.8.8
```

Save: `Ctrl+O` → `Enter` → `Ctrl+X`

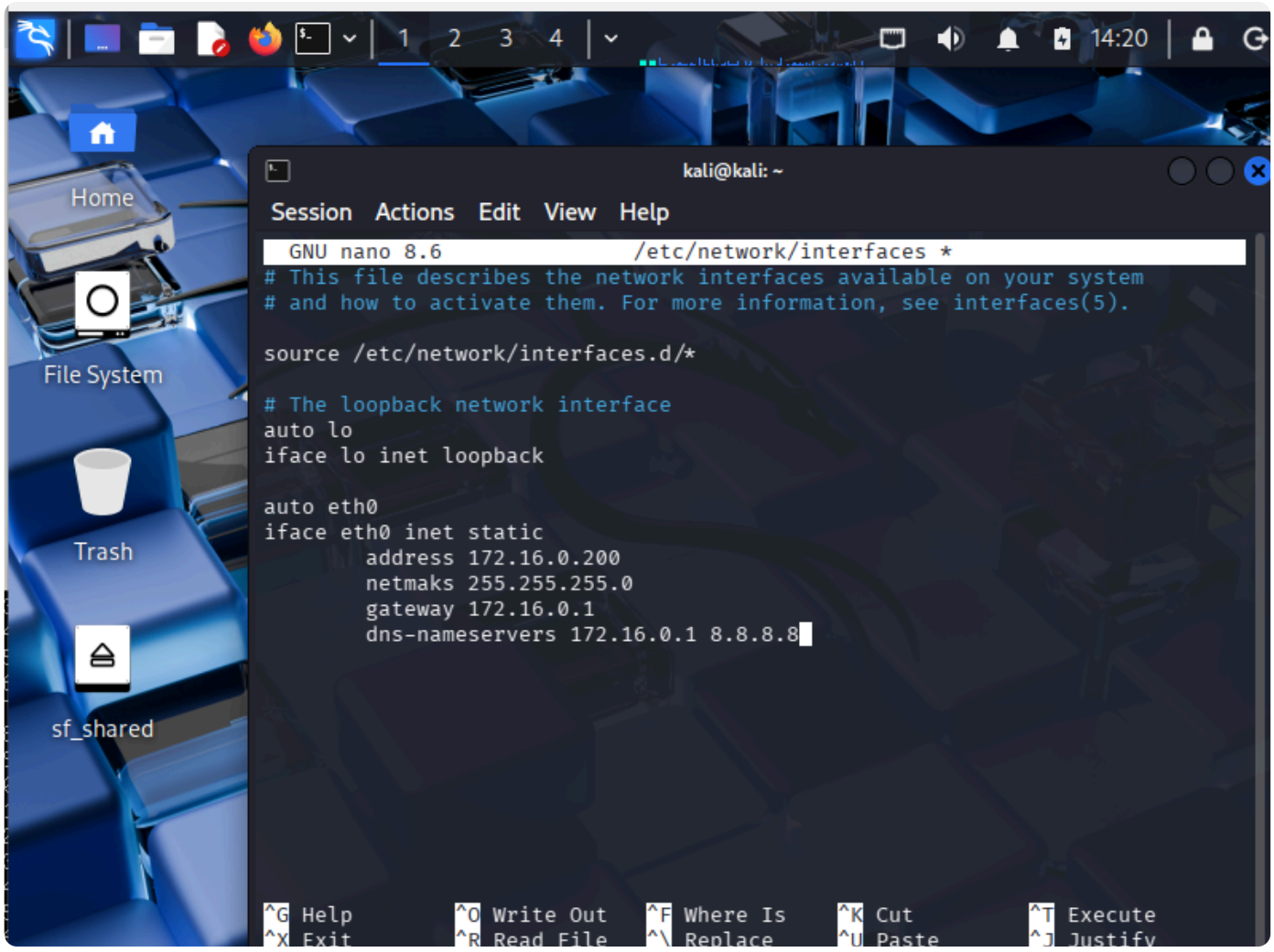
Restart networking:

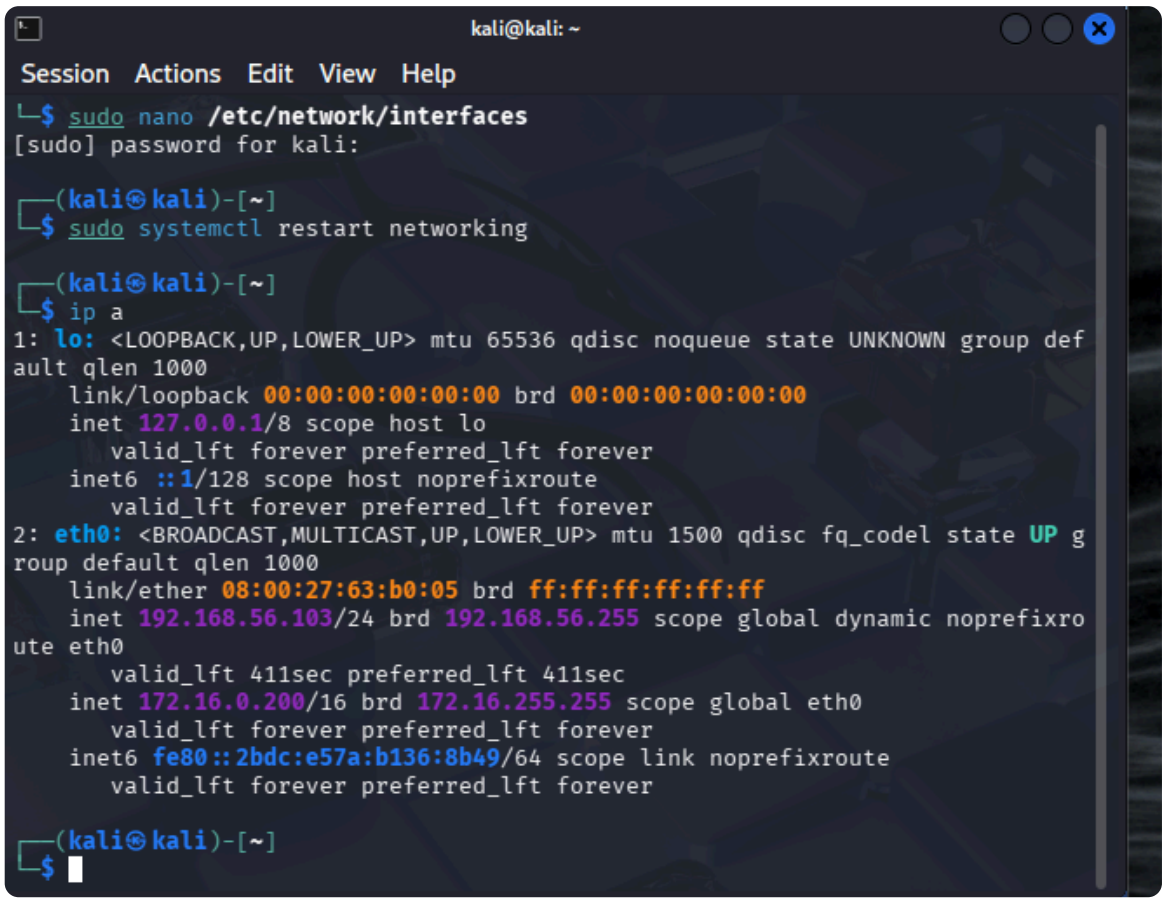
```
sudo systemctl restart networking
# OR
sudo ifconfig eth0 down
sudo ifconfig eth0 up
```

Verify IP:

```
ip a
```

You should see `172.16.0.200/24`


1 2 3 4
14:20
Home
File System
Trash
sf_shared
kali@kali: ~
Session Actions Edit View Help
GNU nano 8.6 /etc/network/interfaces *
This file describes the network interfaces available on your system
and how to activate them. For more information, see interfaces(5).
source /etc/network/interfaces.d/*
The loopback network interface
auto lo
iface lo inet loopback
auto eth0
iface eth0 inet static
address 172.16.0.200
netmaks 255.255.255.0
gateway 172.16.0.1
dns-nameservers 172.16.0.1 8.8.8.8
^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify


kali@kali: ~
Session Actions Edit View Help
└$ sudo nano /etc/network/interfaces
[sudo] password for kali:
┌──(kali㉿kali)-[~]
└$ sudo systemctl restart networking
┌──(kali㉿kali)-[~]
└$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group def
ault qlen 1000
link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
inet 127.0.0.1/8 scope host lo
valid_lft forever preferred_lft forever
inet6 ::1/128 scope host noprefixroute
valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP g
roup default qlen 1000
link/ether 08:00:27:63:b0:05 brd ff:ff:ff:ff:ff:ff
inet 192.168.56.103/24 brd 192.168.56.255 scope global dynamic noprefixro
ute eth0
valid_lft 411sec preferred_lft 411sec
inet 172.16.0.200/16 brd 172.16.255.255 scope global eth0
valid_lft forever preferred_lft forever
inet6 fe80::2bdc:e57a:b136:8b49/64 scope link noprefixroute
valid_lft forever preferred_lft forever
┌──(kali㉿kali)-[~]
└$